Activity 1: Reading a CSV File

A CSV file is a data file that has the file extension .csv. It contains lines of text where each line consists of comma separated values. Typically, the first line consists of headings where as the remaining lines contain the data values. For example, a CSV file containing data related to clothing may be as follows:

```
id,name,colour
1,Shirt,Mercury
2,Coat,Boulder
3,Trousers,Black
```

If we know the structure of a file then often this makes it easier to process the file. In terms of CSV files, we can use the module csv in python to help us read and process the file. We can import the module as follows:

```
import csv
```

We can use the method reader of the module csv to read the file. When used with a loop, the reader object will return a list of values for each line in the file:

```
import csv

with open("data.csv") as file:
  csv_reader = csv.reader(file)
  for values in csv_reader:
    # do something
```

Often, when our file contains headings, we read in and store the headings separately. We can do this by using the python function next(). This function returns the next item i.e. line in a file.

```
import csv

with open("data.csv") as file:
  csv_reader = csv.reader(file)
  headings = next(csv_reader)
  for values in csv_reader:
    # do something
```

[Tasks]

We wish to create a program to display the content of a CSV file.

You should start by creating a suitable CSV file named clothing.csv and store it in the following location: ?

```
week6/clothing.csv
```

The CSV file should contain the following data:

```
id,name,colour
1,Shirt,Mercury
2,Coat,Boulder
3,Trousers,Black
```

Then create a python file called csv_tasks.py inside the folder you created previously (week6).

The program should consist of the following functions:

The first function should be named read_csv and should take 1 parameter which is a file path for the CSV file.
The function should start by opening the specified file for reading.
The function should then read in and display the headings in the format "Headings:\n{headings}" where {headings} is the headings that have been retrieved from the CSV file.
The function should then display the message "Values:".
For each line remaining in the file, the function should do the following:
  The function should display the values for that line.
Finally, the function should close the file if necessary.

The second function should be named run_task1 and should take no parameters.
The function should call the function read and pass it the value clothing.csv as the file_path.

You should also add the following code to the end of your file so that it executes the function run_task1 when the file is executed directly.

```
if __name__ == "__main__":
  run_task1()
```

An example run of such a program is shown below:

```
Headings:
['id', 'name', 'colour']
Values:
['1', 'Shirt', 'Mercury']
['2', 'Coat', 'Boulder']
['3', 'Trousers', 'Black']
```

Code your solution and include any appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to demonstrate how to read a CSV file.".

Activity 2: Extract Values from CSV

After reading in values from a CSV file using the module csv, we can access each individual value using an index. For example, if the first line in our file is "id,name,colour" then the CSV reader will give us the values ["id","name","colour"]. We can get the first value using values[0] which gives "id". We can get the second value using values[1] which gives "name" and so on.

```
import csv
with open("clothing.csv") as file:
  csv_reader = csv.reader(file)
  headings = next(csv_reader)
  for values in csv_reader:
    # display only the IDs
    print(values[0])
```

[Tasks]

We wish to create a program to help us extract values from the CSV file we created in the previous activity i.e. clothing.csv.

The program should consist of the following functions:

A function named extract which takes 1 parameter that represents a file path for our CSV file.
The function should start by displaying the message "Extracting...".
The function should then open the specified file for reading.
The function should read in and ignore the headings.
The function should create a variable named names which is set to an empty string i.e. ""
For each remaining line in the file:
  The function should read in the values and extract the name of the clothing item.
  The function should add the extracted name with a new line character to the variable names.
After all the lines have been processed, the function should display the message "Done! The extracted items are as follows:".
The function should then display the extracted names.
Finally, the function should close the file if necessary.

The second function should be named run_task2 and take no parameters.
The function should call the first function and pass the value "clothing.csv" for the parameter file_path.

An example run of such a program is shown below:

```
Extracting...Done!
The extracted items are:
Shirt
Coat
Trousers
```

Code your solution and be sure to include appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to extract data from the CSV file.".

Activity 3: Export CSV

We previously learnt that we can write to a file by opening a file in write mode and using the method write.

```
with open("output.csv", "w") as file:
  file.write("some,csv,data")
```

If the file does not already exist then a new file will be created otherwise the existing file will be overwritten. We can also append to an existing file by opening the file in append mode. To do this, we specify the mode as "a".

```
with open("output.csv", "a") as file:
  file.write("some,csv,data")
```

To export data to a CSV file we simply open the file in write or append mode and write out the data in the CSV format i.e. use commas to separate values

[Tasks]

We wish to create a program to export user data as a CSV file.

You should start by creating a suitable CSV file named exported_items.csv and store it in the following location:

```
week6/exported_items.csv
```

The CSV file should contain the following:

```
id,name,colour
1,Shirt,Mercury
2,Coat,Boulder
3,Trousers,Black
```

The program should consist of the following functions:

The function should be named export and should take 2 parameters that represent a file path to the CSV file (i.e. exported_items.csv) and an integer indicating how many items are to be exported.
The function should start by displaying the message "Exporting...".
The function should then open the specified file for appending.
For each item to be exported, the function should do the following:
- Ask the user to enter the id of the items and assign the user's response to a variable named item_id.
- Ask the user to enter the name of the item and assign the user's response to a variable named item_name.
- Ask the user to enter the colour for the item and assign the user's response to a variable named item_colour.
- Write the values of the variables to the CSV file in the format "{item_id},{item_name},{item_colour}".

The function should close the file, if necessary.
The function should then display the message "Done!".

A function run which calls the first function with suitable values.

An example run of such a program is shown below.

```
Exporting...
Please enter the item id:
4
Please enter the item name:
Skirt
Please enter the item colour:
Cyan
Please enter the item id:
5
Please enter the item name:
Hat
Please enter the item colour:
Red
Done!
```

The content of the file exported_items.csv should similar to the following if the user enters the values 4, Skirt, Cyan and 5,Hat, Red:

```
id,name,colour
1,Shirt,Mercury
2,Coat,Boulder
3,Trousers,Black
4,Skirt,Cyan
5,Hat,Red
```

Code your solution and be sure to include appropriate comments in your code.

[Commit to Repository]

It is time to commit our working program to our repository. Use appropriate commands to commit your changes. You should add a description for the commit summarising the changes you have made e.g. "Added code to export csv data.".

Originally created by Prins Butt, updated and modified by Darren Cunningham.

Last modified: Saturday, 15 November 2025, 2:27 PM

◀ 2 - Text Files

Jump to...

File Handling in Python ▶

Dashboard / Problem Solving Through Programming (QHO444) / QHO444 Materials / 3 - CSV Files

Study

Succeed@Solent

Referencing
Subject Guides
Library
Ethics

Organise

Email
Timetables
Term Dates
Portal

Support

Student Hub
Technical Support
Printing
Extenuating Circumstances

Solent Futures

Solent Careers Online
CV Help
Placements

Disclaimer | Terms & Conditions | Cookies | Accessibility Statement |

© Solent University